

InstaGrader: An AI-powered Essay Grading Platform

Nathan Fredericks
Jodrey School of Computer Science
Acadia University
Wolfville, NS, Canada
0300722f@acadiau.ca

I. INTRODUCTION

InstaGrader is a web application that helps teachers grade essays. Grading is one of the biggest causes of teacher burnout because it takes so much time and energy. Teachers often spend hours reading through student work, writing feedback, and assigning scores. InstaGrader aims to reduce this workload by using artificial intelligence to provide a first draft of grades and feedback that teachers can then review and adjust.

The application works like this:

- Teachers upload a zip file containing student essays.
- The system reads each essay and scores it based on a rubric set by the teacher.
- Once the essays have been scored, the teacher reviews each one individually.
- The rubric scores and AI-generated comments are displayed, and the teacher can approve, reject, or change the feedback and grade.
- Teachers can mark up the assignment using highlights, strikethroughs, and comments, just like they would on paper.

The goal of InstaGrader is not to replace teachers, but to give them a starting point that saves time while keeping them in full control of the final grade.

II. THE PROBLEM

Grading essays takes a long time. A single essay can take anywhere from five to fifteen minutes to read, score, and give feedback on. When a teacher has thirty or more students per class and multiple classes, the hours add up quickly. This workload contributes to teacher burnout, leading to lower-quality student feedback and higher turnover rates in schools.

Delayed feedback also hurts student learning. When students get their graded essays weeks after submission, they have often moved on to new topics, and the feedback becomes less useful. Faster grading means students can learn from their mistakes while the assignment is still fresh in their minds.

InstaGrader addresses these problems by automating the initial grading process. The AI provides a first pass on each essay, and teachers review and finalize the results. This keeps the human element in grading while reducing the time spent on each essay.

III. TARGET AUDIENCE

The target audience for InstaGrader is teachers who feel overwhelmed with grading. These teachers tend to be at the middle school or high school level, where essay assignments are common, and class sizes can be large. The application is also useful for first-year university classes, where there are many students and essays may be shorter or more straightforward. These situations are good candidates for AI-assisted grading because the volume of work is high, but the essays and rubrics themselves are not overly complex.

IV. COMPARISON TO EXISTING SOLUTIONS

There are a handful of similar products on the market, and they all work in different ways. Some focus on other assignment types, use different AI approaches, and have different interfaces.

A. CoGrader and EssayGrader

CoGrader [1] and EssayGrader [2] are the most similar applications to InstaGrader. They work similarly by running every essay through an AI model to generate feedback. CoGrader and EssayGrader both use a review-first approach that requires teachers to review grades before exporting them and allows teachers to edit the feedback given to students.

The main differences between InstaGrader and these competitors are:

a) *Custom AI Model With Privacy Built-In:*

Instead of using a commercial AI provider, I am fine-tuning my own model using open-source tools. This gives me full control over how the model behaves and protects student data privacy, since no essay content is sent to any external companies.

b) *Paper-Like Annotation Experience:*

InstaGrader allows teachers to mark up assignments just like they would on paper, with highlights, strikethroughs, and comments directly on the essay text. This workflow makes the move from physical to digital grading feel natural and can make the application easier to use for teachers who are less comfortable with technology.

V. TECHNOLOGY

This section describes the tools and systems that will be used to build InstaGrader.

A. How the System Works

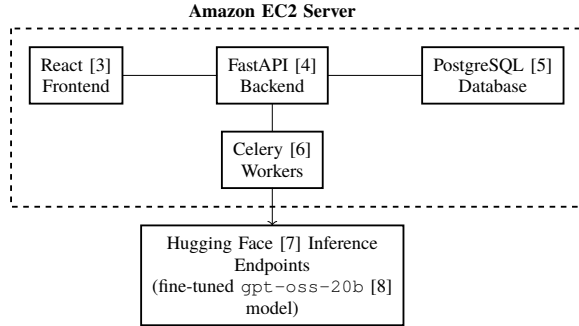


Fig. 1. System Architecture Overview

As shown in Fig. 1, the user interacts with the website (built with React [3]), which communicates with the backend server (built with FastAPI [4]). When essays are uploaded, the backend stores them and adds grading tasks to a queue. Celery [6] is a tool for handling background tasks so users do not have to wait for long-running processes to finish. The workers use MarkItDown [9] to extract text from the uploaded files, then send the essay and rubric to the AI service. The grading results are stored in a PostgreSQL [5] database and shown to the teacher for review.

B. Frontend

The frontend will be built using React [3] and TypeScript. I chose these because they are simple to work with and let me build quickly. The look and feel will use Material UI [10], a set of pre-built components. Material UI is accessible and will look familiar to teachers who already use Google products.

For displaying essays with annotations, I will use ProseMirror [11], which is actively maintained and can be customized to fit my needs.

C. Backend

The backend will be built with Python using FastAPI [4]. I chose FastAPI because it is lightweight and well-suited to handling many concurrent requests.

The database will be PostgreSQL [5]. I am familiar with it, and it works well for storing structured data, such as rubrics and grading results.

Since the application needs to handle many tasks at once, I will use Celery [6] with Valkey [12] to process work in the background. This includes extracting text from uploaded files, sending documents to the AI, and processing the results. Celery can recover from errors, retry failed tasks, and distribute work across multiple processes.

To extract text from documents, I will use Microsoft's MarkItDown [9] project. MarkItDown can convert almost any document type into a simple text format that the AI can understand.

D. AI Model and Training

Instead of using a commercial AI model, I am training my own model with open-source tools. I am doing this for two reasons:

a) Protect Student Data Privacy:

Student essays will not be sent to outside AI services. All processing happens on servers I control.

b) Learn About AI Fine-tuning:

I want to understand what goes into training an AI model and how to run it myself.

I will use gpt-oss-20b [8] as my starting point. This is an open-source language model released by OpenAI [13] that anyone can use and modify. It is smaller than the largest commercial models, making it cheaper and faster to run.

For training, I will use Unsloth [14], a tool that enables training AI models on regular computers without expensive hardware. Unsloth has good documentation and works with the latest models.

E. Training Data

I will use the ASAP++ [15] dataset for training my model. ASAP++ is based on the original ASAP [16] dataset, which was released in 2012. The original ASAP dataset comprises around 13,000 essays across eight prompts, covering narrative and argumentative writing styles. Each essay was scored by human graders, which gives me a foundation to build on.

ASAP++ improves this dataset by adding scores for specific writing traits: Content, Organization, Word Choice, Sentence Fluency, Conventions, and Voice. This lets the model learn to give detailed feedback on different aspects of writing, not just an overall grade.

F. Hosting

To run the AI model, I will use Hugging Face's [7] Inference Endpoints. This is a service that runs AI models in the cloud, which is more cost-effective than running a server continuously.

For hosting the website and backend, I will use an Amazon EC2 [17] instance running Ubuntu [18]. I am familiar with AWS services and already have some experience running servers there. The server will run the website, the backend, the database, and the background workers. The AI model runs separately on the Hugging Face service.

VI. FEATURES

A. Annotation Workflow

The AI will add annotations to the essay that teachers can edit. Teachers can add new annotations on top of the AI suggestions, remove annotations they disagree with, or make changes themselves. This allows teachers to revise the AI feedback to match their own grading style while saving time on the initial markup.

B. Rubric System

Rubrics will be stored in the database in a structured format. Teachers will use a rubric builder within the application to create their rubrics. They will not be able to upload existing rubrics from other systems, but must create their own using the builder. This makes sure that all rubrics are in a consistent format that the AI model can understand and grade against.

VII. PRIVACY AND ETHICS

A. Privacy

The application is designed with privacy in mind. Because I am using a custom model hosted on my own infrastructure, no student essay data is sent to third-party commercial AI providers.

None of the uploaded essays will be used for training the AI model after deployment. Student data will be deleted within 30 days of grading completion and will not be retained beyond that period. Teachers will have control over when their assignments and student essays are removed from the system.

B. Ethics

a) Preventing over-reliance on AI grading:

A human teacher must approve every grade and every change made by the AI. The system will not allow grades to be exported until a teacher has reviewed each essay individually. This keeps teachers in control and ensures that AI is a tool, not a replacement.

b) Bias in grading:

There may be bias in the AI grading that reflects bias in the training data. This will be made clear on the website so that teachers know what to expect and can watch for potential bias in the AI suggestions.

VIII. GOALS

A. Base Goals

These are the minimum requirements to complete the project:

- A working AI model that can read an essay and a rubric, then output scores and feedback
- A basic website where teachers can upload essays, enter a rubric, and view results
- A backend that handles file uploads, converts documents to text, and processes grading
- The application running on a server

B. Should Happen Goals

These are what I am aiming for:

- User accounts with login and registration
- A dashboard to view past assignments
- Batch upload for grading an entire class at once
- An essay viewer that shows annotations right on the text (not just a list)
- A visual rubric builder
- Export grades to CSV and feedback to PDF

C. Stretch Goals

If everything goes well and I have extra time:

- Connect to Google Classroom [19] or other learning management systems
- Let the AI search course materials for context when grading
- Compare results with commercial AI models
- Support for scanned PDFs using text recognition

IX. CONCERNS AND RISKS

A. AI Model Accuracy

The smaller open-source model may not grade as well as larger commercial models. If the AI gives unreliable suggestions, teachers may not trust or use the system. To address this, I will start training early and test the model often.

B. Fallback Plan

If fine-tuning does not produce acceptable results, I will fall back to using the base model with detailed system prompt. This approach trades some accuracy for reliability and can be implemented quickly. The rubric and essay will be formatted into a structured prompt that guides the model to output scores in a consistent format.

C. Fine-tuning Takes Multiple Tries

AI models rarely work perfectly on the first attempt. I have set aside two full weeks for training and testing the model.

D. Annotation Viewer Complexity

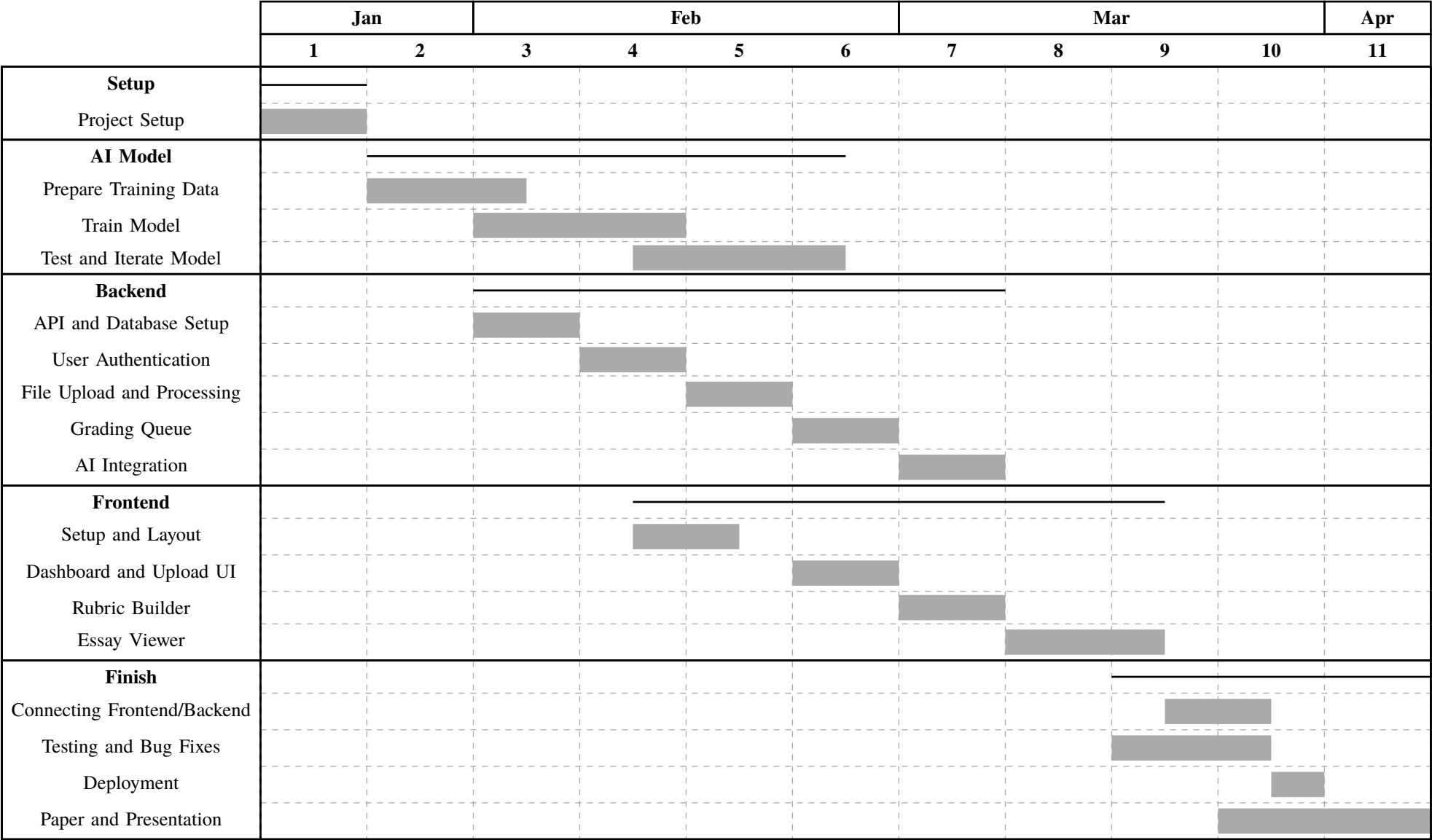
Showing annotations directly on essay text is technically challenging. I will start simple and build up, using existing tools where possible.

REFERENCES

- [1] "CoGrader." Accessed: Jan. 19, 2026. [Online]. Available: <https://cograder.com/>
- [2] "EssayGrader." Accessed: Jan. 20, 2026. [Online]. Available: <https://essaygrader.ai/>
- [3] "React." Accessed: Jan. 20, 2026. [Online]. Available: <https://react.dev/>
- [4] "FastAPI." Accessed: Jan. 20, 2026. [Online]. Available: <https://fastapi.tiangolo.com/>
- [5] "PostgreSQL." Accessed: Jan. 20, 2026. [Online]. Available: <https://www.postgresql.org/>
- [6] "Celery." Accessed: Jan. 21, 2026. [Online]. Available: <https://docs.celeryq.dev/>
- [7] "Hugging Face." Accessed: Jan. 19, 2026. [Online]. Available: <https://huggingface.co/>
- [8] "gpt-oss-20b." Accessed: Jan. 21, 2026. [Online]. Available: <https://huggingface.co/openai/gpt-oss-20b>
- [9] "MarkItDown." Accessed: Jan. 19, 2026. [Online]. Available: <https://github.com/microsoft/markitdown>
- [10] "Material UI." Accessed: Jan. 20, 2026. [Online]. Available: <https://mui.com/>
- [11] "ProseMirror." Accessed: Jan. 20, 2026. [Online]. Available: <https://prosemirror.net/>
- [12] "Valkey." Accessed: Jan. 21, 2026. [Online]. Available: <https://valkey.io/>
- [13] "OpenAI." Accessed: Jan. 19, 2026. [Online]. Available: <https://openai.com/>
- [14] "Unsloth AI." Accessed: Jan. 19, 2026. [Online]. Available: <https://unsloth.ai/>
- [15] S. Mathias and P. Bhattacharyya, "ASAP++: Enriching the ASAP Automated Essay Grading Dataset with Essay Attribute Scores," in

Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018), 2018. [Online]. Available: <https://aclanthology.org/L18-1187/>

- [16] “Automated Student Assessment Prize (ASAP) - Automated Essay Scoring.” Accessed: Jan. 21, 2026. [Online]. Available: <https://www.kaggle.com/competitions/asap-aes/overview>
- [17] “Amazon Web Services.” Accessed: Jan. 21, 2026. [Online]. Available: <https://aws.amazon.com/>
- [18] “Ubuntu.” Accessed: Jan. 21, 2026. [Online]. Available: <https://ubuntu.com/>
- [19] “Google Classroom.” Accessed: Jan. 21, 2026. [Online]. Available: <https://classroom.google.com/>



Start
Jan 19

End
Apr 10